МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Посохина Ульяна Евгеньевна
(подпись) (фамилия, имя, отчество)

Группы Ипо-42

Руководитель практической подготовки от профильной организации

_____
(должность)

_____ _____
(подпись) (фамилия, имя, отчество)

МП «16» марта 2026 года

Руководитель практической подготовки от института

_____ _____
(подпись) (фамилия, имя, отчество)

«16» марта 2026 года

Пермь, 2026 год

Содержание

Введение 2
Раздел 1. Организационно подготовительный этап прохождения практики на предприятии 5
Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии 5
Раздел 2. Ознакомление со структурой и характером деятельности подразделения 8
Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении 8
Тема 2.2. Ознакомление с должностными и функциональными обязанностями 10
Тема 2.3. Организация информационного обеспечения и защиты информации предприятия 12
Тема 2.4. Структурные и функциональные схемы предприятия 13
Раздел 3. Работа на рабочих местах или в подразделениях предприятиях 17
Тема 3.1. Выполнение производственных заданий 17
Заключение 35
Список используемых источников 36
Приложения 39

Введение

Практика имеет целью комплексное освоение студентами всех видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование», формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности.

Производственная практика направлена на освоение обучающимися профессиональных компетенций в процессе самостоятельного выполнения определенных видов работ, необходимых для осуществления профессиональной деятельности в максимально приближенных к производству условиях, в рамках модулей ППССЗ по основным видам профессиональной деятельности для освоения методов и приемов практического применения прикладных программных продуктов для программного обеспечения компьютерных систем.

В настоящей работе представлен отчет о прохождении производственной практики студента-практиканта. Основными видами деятельности которой являлись: формирование требований к программным модулям на основании анализа проектной и технической документации с целью обеспечения корректного взаимодействия компонентов; описание процесса моделирования разработки ПО. Задачи практики (освоение обучающимися профессиональных и общих компетенций):

1. Выполнять интеграцию модулей в программное обеспечение;
2. Выполнять отладку программного модуля с использованием специализированных программных средств;
3. Осуществлять разработку тестовых наборов и тестовых сценариев для программного обеспечения;
4. Производить инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;

5. Выбирать способы решения задач профессиональной деятельности, применительно к различным контекстам;

6. Осуществлять поиск, анализ и интерпретацию информации, необходимой для выполнения задач профессиональной деятельности

7. Планировать и реализовывать собственное профессиональное и личностное развитие;

8. Работать в коллективе и команде, эффективно взаимодействовать с коллегами, руководством, клиентами

9. Осуществлять устную и письменную коммуникацию на государственном языке Российской Федерации с учетом особенностей социального и культурного контекста.

10. Использовать информационные технологии в профессиональной деятельности

11. Пользоваться профессиональной документацией на государственном и иностранном языках.

Раздел 1. Организационно подготовительный этап прохождения практики на предприятии

Тема 1.1. Инструктаж по прохождению производственной практики и правилам безопасности работы на предприятии

В период прохождения практики обучающийся обязан: строго соблюдать правила внутреннего трудового распорядка предприятия; выполнять задания, порученные руководителем практики от организации, в установленные сроки; соблюдать последовательность технологических операций, предусмотренных регламентом работы; своевременно информировать руководителя о возникающих трудностях и нештатных ситуациях; вести дневник практики, фиксируя выполненные работы и приобретённые навыки; соблюдать конфиденциальность информации, полученной в ходе работы; поддерживать чистоту и порядок на рабочем месте; проходить все предусмотренные инструктажи по охране труда и технике безопасности. По окончании практики обучающийся должен представить отчет о прохождении практики, оформленный в соответствии с установленными требованиями предприятия и образовательной организации. Отчет должен содержать подробное описание выполненных работ, анализ приобретенных навыков и знаний, а также выводы и предложения по совершенствованию производственных процессов.

Обязательны к исполнению: обязательное соблюдение знаков безопасности и предупредительных табличек; знание местоположения аптечки первой помощи и средств пожаротушения; умение действовать при пожаре, аварии или иной чрезвычайной ситуации (знать пути эвакуации, номера экстренных служб); немедленное информирование руководителя о травмах, недомогании или неисправностях оборудования; соблюдение правил личной гигиены (приём пищи только в отведённых местах, мытьё рук перед едой). Помимо неукоснительного следования общим правилам, работники обязаны проявлять бдительность и ответственность в отношении

своей собственной безопасности и безопасности окружающих. Это включает в себя ношение и использование средств индивидуальной защиты (СИЗ), если это требуется спецификой выполняемой работы. Важно понимать, что Особое внимание необходимо уделять безопасному использованию оборудования и инструментов. Перед началом работы следует убедиться в их исправности и соответствии требованиям безопасности. Недопустимо использование неисправного оборудования или инструментов, а также выполнение работ, для которых они не предназначены. В случае обнаружения неисправностей или сомнений в правильности работы оборудования следует немедленно сообщить об этом руководителю. Важным аспектом является поддержание порядка и чистоты на рабочем месте. Загроможденные проходы, разбросанные инструменты и материалы могут создать опасные ситуации и привести к травмам. Необходимо своевременно убирать мусор, следить за чистотой пола и обеспечивать свободный доступ к оборудованию и инструменту. Помимо прочего, каждый работник должен быть осведомлен о потенциальных опасностях и рисках, связанных с выполняемой работой, и о мерах по их предотвращению. Регулярное прохождение инструктажей и тренингов по охране труда является обязательным условием для обеспечения безопасной рабочей среды. Знания, полученные на этих мероприятиях, помогут работникам правильно оценивать риски и принимать меры по их минимизации. Соблюдение правил техники безопасности – это не просто требование, а личная ответственность каждого работника. От этого зависит не только его собственное здоровье и благополучие, но и безопасность его коллег.

До начала работы: проверить целостность изоляции проводов, штепсельных вилок, розеток и корпусов приборов; убедиться в отсутствии повреждений на корпусе прибора, оголённых токоведущих частей; проверить наличие заземления (при необходимости); осмотреть рабочее место на отсутствие влаги, горючих материалов, посторонних предметов; надеть диэлектрические перчатки и другие требуемые СИЗ.

Во время работы: включать приборы только сухими руками, держась за корпус вилки; не вытягивать вилку из розетки за шнур, а аккуратно вытягивать за корпус вилки; не перегружать электросеть, не использовать повреждённые удлинители; не ремонтировать электроприборы самостоятельно, не вскрывать корпуса под напряжением; не прикасаться к работающим приборам мокрыми руками или влажными предметами; не оставлять включённое оборудование без надзора; при работе с измерительными приборами следовать инструкции по эксплуатации, не превышать допустимые пределы измерений; при замене элементов питания соблюдать полярность и использовать только рекомендованные батареи. В аварийных ситуациях: при появлении запаха гари, искрении, перегреве прибора — немедленно отключить его от сети; при поражении человека электрическим током — отключить питание, оказать первую помощь, вызвать медперсонал; при возгорании электроприбора — отключить питание, использовать огнетушитель (не водой!), вызвать пожарную службу; при обнаружении обрыва проводов или повреждения электрооборудования — не приближаться, оповестить ответственного за электробезопасность.

По окончании работ: отключить приборы от сети; убрать кабели и провода, не оставлять их на проходе; проверить отсутствие нагрева корпусов и запахов; сдать оборудование на хранение в установленное место.

Раздел 2. Ознакомление со структурой и характером деятельности подразделения

Тема 2.1. Ознакомление с организацией работы на предприятии или в структурном подразделении

2.1.1 Структура предприятия и ИТ-подразделения

ООО «АРУС ПАРУС» зарегистрировано 08.08.2024 в г. Перми. Организационная структура — линейно-функциональная.

Высший орган управления — Общее собрание участников. Состав учредителей:

Сабуров А.Л. — 34% уставного капитала;

Карпов С.В. — 33%;

Харгелия О.Р. — 33%.

Единоличный исполнительный орган — генеральный директор Харгелия О.Р.

Таблица 1 – Функции персонала

Функциональное направление	Основные функции
Административно-управленческий персонал	Общее руководство, стратегическое планирование, взаимодействие с заказчиками, финансовое и кадровое обеспечение
Проектный отдел (разработка ПО)	Разработка ПО (ОКВЭД 62.01)
Отдел консультирования и сопровождения	Консультативные услуги (ОКВЭД 62.02), управление оборудованием (ОКВЭД 62.03)
Отдел обработки данных	Обработка данных, размещение информации (ОКВЭД 63.11), web-порталы (ОКВЭД 63.12)

Ввиду статуса микропредприятия отдельное ИТ-подразделение не выделено. Функции разработки и сопровождения распределены между проектными командами под конкретные задачи.

2.1.2 Функции подразделений и их взаимодействие

Взаимодействие:

Административно-управленческий персонал распределяет заказы по проектным группам;

Проектный отдел передает ПО в отдел сопровождения;

Отдел консультирования формирует требования на доработку;

Отдел обработки данных обеспечивает инфраструктуру.

Компания связана с ООО «АРУС ТЕХНОЛОГИЯ», ООО «ФАРРО-ТМ», ООО «НПП «АКВАСИНТ» через учредителей.

2.1.3 Режим работы и формы организации труда

Режим:

5-дневная неделя, 8-часовой день (9:00–18:00), перерыв 13:00–14:00.

Формы труда:

гибкий график для инженеров;

проектная организация;

удаленная работа;

электронный учет времени;

оперативные совещания.

2.1.4 Краткая характеристика предприятия

Таблица 2 – Характеристика предприятия

Параметр	Значение
Полное наименование	ООО «АРУС ПАРУС»
Дата регистрации	08.08.2024

Параметр	Значение
Юридический адрес	г. Пермь, ул. Пушкарская, д. 98, кв. 75
ИНН/КПП	5906181316 / 590601001
ОГРН	1245900013226
Уставный капитал	100 000 руб.
Основной вид деятельности	Разработка ПО (ОКВЭД 62.01)
Руководитель	Харгелия О.Р.
Категория МСП	Микропредприятие (с 10.09.2024)
Система налогообложения	УСН

Основные направления: разработка ПО, АСУ ТП, консультационные услуги в сфере IT. С апреля 2025 года — член Ассоциации проектировщиков «Развитие проектной отрасли».

Тема 2.2. Ознакомление с должностными и функциональными обязанностями

2.2.1 Изучение должностной инструкции сотрудника ИТ-подразделения

Изучена должностная инструкция инженера-программиста (специалиста по АСУ ТП).

Общие положения:

категория — специалист;

высшее техническое образование без требований к стажу или среднее профессиональное с опытом;

подчинение руководителю проектного отдела;

назначение и освобождение — приказом гендиректора.

Квалификационные требования: знание нормативных материалов, средств вычислительной техники, порядка разработки документации, правил эксплуатации, методов оценки эффективности, охраны труда.

2.2.2 Права и обязанности специалиста

Таблица 3 – Должностные обязанности

Направление	Обязанности
Разработка ПО	Разработка по ТЗ, алгоритмы, программирование ПЛК (МЭК 61131-3), HMI/SCADA, тестирование
Сопровождение	Поддержка проектов, устранение отказов, техобслуживание, доработка
Документирование	Техдокументация, инструкции, отчетность
Работа с оборудованием	Чтение схем, подбор оборудования, пусконаладка

Права: знакомство с проектами решений, внесение предложений, запрос информации, привлечение специалистов, требование условий для работы.

2.2.3 Согласование индивидуального задания

Согласовано задание:

Сопровождение ПО автоматизации;

Ознакомление со SCADA и средами разработки ПЛК;

Тестирование и отладка;

Работа с документацией;

Сбор материалов для отчета.

2.2.4 Цели и задачи практики

Цель: получение опыта сопровождения ПО и изучение информационных процессов в промышленной автоматизации.

Задачи:

изучение структуры предприятия и работы отдела АСУ ТП;

анализ средств автоматизации;

настройка специализированного ПО;

работа с базами данных;

изучение методов защиты информации;

сбор материалов для отчета.

Ожидаемые результаты: закрепление знаний, формирование навыков, опыт проектной работы, развитие компетенций.

Тема 2.3. Организация информационного обеспечения и защиты информации предприятия

Используемые программные средства:

Среды разработки ПЛК: CODESYS V3.5, TIA Portal, Unity Pro;

SCADA-системы: MasterSCADA 4D, Alpha.Platform, Simatic WinCC;

САПР: sPlan, AutoCAD Electrical, EPLAN Electric P8;

СУБД: MS SQL Server, PostgreSQL;

Системы управления проектами: Git, Redmine/YouGile, СЭД;

Инструменты моделирования: AnyLogic, MATLAB/Simulink.

2.3.1 Процессы сбора, регистрации и обработки информации

Сбор: данные с ПЛК и КИП, техническая документация от заказчиков. Регистрация: параметры в БД, коды в Git, документация в СЭД. Обработка: визуализация в SCADA, анализ, отчеты, архивация.

2.3.2 Методология проектирования и эксплуатации

Проектирование: каскадный подход с элементами итераций (анализ — проектирование — разработка — тестирование — внедрение — сопровождение).

Эксплуатация: мониторинг, регламентное обслуживание, управление инцидентами и изменениями.

2.3.3 Методы защиты информации

Организационные меры: ролевая модель, политика конфиденциальности, инструктаж, регламенты.

Таблица 4 – Технические средства защиты

Уровень защиты	Средства
Защита рабочих станций	Kaspersky Endpoint Security, контроль запуска программ
Сетевая защита	Межсетевые экраны, прокси, фильтрация трафика
Защита доступа	Active Directory, индивидуальные логины/пароли
Защита от перебоев	APC Smart-UPS
Резервное копирование	Veeam Backup, Acronis, RAID
Удаленный доступ	VPN

Программно-технические меры: WSUS, лицензионное ПО, логирование, контроль целостности. Правовые меры: соблюдение законодательства, лицензии, договоры о конфиденциальности.

Тема 2.4. Структурные и функциональные схемы предприятия

2.4.1 Требования к техническим и программным средствам

Таблица 5 – Аппаратное обеспечение

Компонент	Минимум	Фактически
Процессор	Intel Core i5	Intel Core i7-12700
ОЗУ	16 ГБ	32 ГБ DDR4
Накопитель	SSD 256 ГБ	NVMe SSD 1 ТБ
Видеокарта	Дискретная, 2 ГБ	RTX 3060
Мониторы	1×22"	2×27" IPS
Сеть	100 Мбит/с	Гигабит
ИБП	—	APC Smart-UPS 1500 BA

Таблица 6 – Программное обеспечение

Категория	ПО
ОС	Windows 11 Pro Workstations
Офис	Microsoft Office Professional Plus
Разработка ПЛК	CODESYS, TIA Portal, Unity Pro
SCADA	MasterSCADA, Alpha.Platform, Simatic WinCC
САПР	sPlan, AutoCAD Electrical, EPLAN
СУБД	MS SQL Server, PostgreSQL
Антивирус	Kaspersky Endpoint Security

Категория	ПО
Резервное копирование	Veeam Backup, Acronis

2.4.2 Анализ средств защиты информации

Организационные: ролевая модель, политика, инструктаж, договоры. Программно-технические (Таблица 7):

Уровень защиты	Средства
Сетевая защита	Межсетевые экраны, прокси, VPN
Защита станций	Kaspersky, контроль программ, WSUS
Защита данных	Резервное копирование, RAID, шифрование
Контроль доступа	Active Directory, логины/пароли

Инженерно-технические: ИБП, сетевые фильтры, заземление, контроль кабелей.

2.4.3 Делопроизводство и ИТ-документация

Таблица 8 – Виды документации

Тип	Форма
Техническая	СЭД, бумажный архив
Проектная (ТЗ, чертежи)	СЭД, файлы САПР
Программная	Git, файлы
Эксплуатационная	PDF, бумажная
Отчетная	СЭД, печать
Организационно-	Электронная, бумажная

Тип	Форма
распорядительная	

Порядок: регистрация, хранение с резервированием, учет версий в Git, разграничение доступа, архивация.

2.4.4 Применяемые технологии безопасности

Таблица 9 – Технологии безопасности

Технология	Назначение	Реализация
Межсетевое экранирование	Защита от НСД	Межсетевой экран
Прокси-сервер	Фильтрация трафика	Прокси-сервер
VPN	Удаленный доступ	VPN-сервер
Антивирус	Защита от вредоносного ПО	Kaspersky
Централизованное обновление	Управление обновлениями	WSUS
Резервное копирование	Защита данных	Veeam, Acronis
RAID-массивы	Отказоустойчивость	Файловые серверы
ИБП	Защита питания	APC Smart-UPS
Политика паролей	Защита доступа	Требования к сложности

Раздел 3. Работа на рабочих местах или в подразделениях предприятиях

Тема 3.1. Выполнение производственных заданий

3.1.1. использование выбранной системы контроля версий;

Использование системы контроля версий при разработке программного продукта

В процессе выполнения практических задач при создании программного обеспечения использовалась распределенная система управления версиями Git. Этот инструмент обеспечивает отслеживание изменений, вносимых в исходный код, позволяет нескольким разработчикам одновременно работать над проектом и хранит полную историю модификаций, что дает возможность при необходимости возвращаться к более ранним версиям файлов.

Практическая работа с Git включала несколько основных операций: создание локального репозитория на рабочей станции, фиксацию промежуточных результатов с помощью коммитов, создание отдельных веток для реализации различных функций проекта и последующую синхронизацию с удаленным хранилищем. Для гарантированного сохранения результатов и обеспечения доступа руководителя практики к материалам локальный репозиторий был связан с удаленным репозиторием, размещенным на платформе GitHub.

Управление версиями выполнялось двумя способами: с использованием встроенных средств графического интерфейса среды Visual Studio 2022, а также через командную строку при необходимости выполнения операций слияния веток и разрешения конфликтов. Применение Git позволило обеспечить надлежащий контроль версий разрабатываемого программного продукта и соблюдение требований, предъявляемых к организации совместной работы над проектом.

3.1.2. разработка программного обеспечения;

Методология разработки программного обеспечения

В основе создания программного продукта лежала итерационная модель разработки. Данный подход был выбран в связи с необходимостью поэтапного построения системы, позволяющего вносить коррективы по завершении каждого цикла. Итеративный характер работы дал возможность последовательно создать модули справочников партнеров, номенклатуры продукции и учета реализованных товаров. По окончании каждой итерации проводилась проверка реализованных функций, после чего при необходимости выполнялась доработка с учетом выявленных уточнений. Применение итерационной модели, в отличие от жесткой каскадной схемы, обеспечило возможность корректировки функциональных характеристик системы непосредственно в процессе разработки.

Набор используемых инструментов

При реализации проекта применялся следующий состав программных средств:

- Система управления версиями Git и платформа GitHub – распределенная система для отслеживания изменений в коде и веб-сервис для размещения удаленного репозитория, обеспечивающие коллективную работу над проектом.
- Среда разработки Visual Studio 2022 – основной инструмент для написания, отладки и компиляции программного кода.
- Система управления базами данных PostgreSQL – реляционная СУБД, используемая для хранения и обработки сведений о контрагентах, товарах и совершенных продажах.
- Инструмент администрирования pgAdmin – графическое приложение для управления базой данных PostgreSQL.
- Менеджер пакетов NuGet – встроенное средство для подключения и обновления внешних библиотек, необходимых в ходе разработки.

Технологическая платформа и средства разработки

Реализация программного проекта велась в интегрированной среде Microsoft Visual Studio 2022 с использованием платформы .NET Framework 4.8. Основным языком программирования выбран C#, для построения пользовательского интерфейса задействована технология WPF. Доступ к данным организован через Entity Framework Core – объектно-реляционный преобразователь, позволяющий выполнять запросы к базе данных PostgreSQL с использованием LINQ и работать с бизнес-сущностями в объектно-ориентированном стиле.

Архитектурное представление приложения

В процессе проектирования была создана диаграмма классов, отражающая структуру программного продукта (рисунок 1). Основными элементами архитектуры выступают:

- классы, описывающие объекты предметной области (Partner, Product, Sale, PartnerType);
- промежуточные классы (PartnerViewModel, SaleViewModel), осуществляющие связь между данными и элементами интерфейса;
- контекст базы данных AppDbContext, отвечающий за управление подключениями и формирование запросов.

На диаграмме показаны ключевые взаимосвязи между компонентами, включая отношение «один ко многим» между контрагентом и выполненными продажами, а также между типом партнера и перечнем партнеров. Представленная структура позволяет наглядно проследить логику взаимодействия объектов в рамках создаваемой системы.

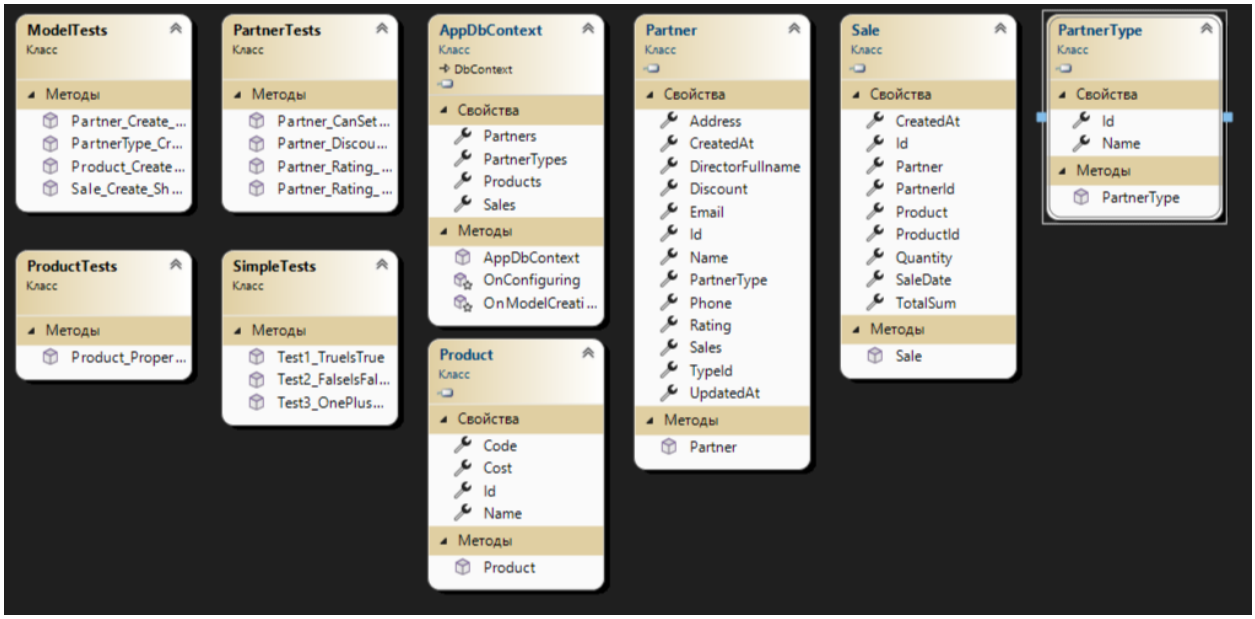

Рисунок 1 – Диаграмма классов программного продукта

```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace PosohinaLibrary.Models
{
    [Table("partners_posohina", Schema = "business")]
    Ссылок 17
    public class Partner
    {
        [Key]
        [Column("id")]
        Ссылок 6
        public int Id { get; set; }

        [Column("type_id")]
        Ссылок 4
        public int TypeId { get; set; }

        [Required]
        [Column("name")]
        [MaxLength(200)]
        Ссылок 6
        public string Name { get; set; }

        [Column("director_fullname")]
        [MaxLength(200)]
        Ссылок 4
        public string DirectorFullname { get; set; }

        [Column("phone")]
        [MaxLength(20)]
        Ссылок 4
        public string Phone { get; set; }

        [Column("email")]
        [MaxLength(100)]
        Ссылок 3
        public string Email { get; set; }


        [Column("address")]
        Ссылок 3
        public string Address { get; set; }


        [Column("rating")]
        Ссылок 8
        public int? Rating { get; set; }

        [Column("discount")]
        Ссылок 6
        public int Discount { get; set; }
```

Рисунок 2 - Код класса «Partner»

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Storage.ValueConversion;
using PosohinaLibrary.Models;
using System;

namespace PosohinaLibrary
{

    public class AppDbContext : DbContext
    {
        Ссылок: 1
        public DbSet<PartnerType> PartnerTypes { get; set; }
        Ссылок: 3
        public DbSet<Partner> Partners { get; set; }
        Ссылок: 1
        public DbSet<Product> Products { get; set; }
        Ссылок: 5
        public DbSet<Sale> Sales { get; set; }

        Ссылок: 0
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseNpgsql("Host=localhost;Database=posohina;Username=app3;Password=123456789;Encoding=UTF8;
            optionsBuilder.EnableSensitiveDataLogging();
            optionsBuilder.EnableDetailedErrors();
        }

        Ссылок: 0
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            modelBuilder.HasDefaultSchema("business");

            modelBuilder.Entity<PartnerType>().ToTable("partner_types_posohina");
            modelBuilder.Entity<Partner>().ToTable("partners_posohina");
            modelBuilder.Entity<Product>().ToTable("products_posohina");
            modelBuilder.Entity<Sale>().ToTable("sales_posohina");

            modelBuilder.Entity<PartnerType>(entity =>
            {
                entity.HasKey(e => e.Id);
                entity.Property(e => e.Id).HasColumnName("id");
                entity.Property(e => e.Name).HasColumnName("name").IsRequired().HasMaxLength(100);
            });

            modelBuilder.Entity<Partner>(entity =>
            {
                entity.HasKey(e => e.Id);
                entity.Property(e => e.Id).HasColumnName("id");
                entity.Property(e => e.TypeId).HasColumnName("type_id");
                entity.Property(e => e.Name).HasColumnName("name").IsRequired().HasMaxLength(200);
                entity.Property(e => e.DirectorFullname).HasColumnName("director_fullname").HasMaxLength(200);
                entity.Property(e => e.Phone).HasColumnName("phone").HasMaxLength(20);
```

Рисунок 3 - Часть контекста базы данных

```
using PosohinaLibrary.Models;
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace PosohinaApp.ViewModels
{
    Ссылок: 9
    public class PartnerViewModel : INotifyPropertyChanged
    {
        private Partner _partner;

        Ссылок: 1
        public PartnerViewModel(Partner partner)
        {
            _partner = partner;
        }

        Ссылок: 7
        public int Id => _partner.Id;
        Ссылок: 2
        public string DisplayName => $"{_partner.PartnerType?.Name} / {_partner.Name}";
        Ссылок: 1
        public string Director => _partner.DirectorFullname ?? "Не указан";
        Ссылок: 1
        public string Phone => _partner.Phone ?? "Не указан";
        Ссылок: 0
        public int? Rating => _partner.Rating;
        Ссылок: 1
        public int Discount => _partner.Discount;


        Ссылок: 0
        public string Address => _partner.Address ?? "Не указан";


        Ссылок: 2
        public Partner Partner => _partner;

        public event PropertyChangedEventHandler PropertyChanged;
        Ссылок: 0
        protected void OnPropertyChanged([CallerMemberName] string name = null)
        {
            PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));
        }
    }
```

Рисунок 4 - Часть программного сервиса «PartnerService»

Разработка программного модуля и этапы создания приложения

Реализация функциональных возможностей

В соответствии со спецификациями, приведенными в приложении 1, была осуществлена разработка программного модуля. В ходе выполнения работ реализованы следующие функции: отображение списка контрагентов, добавление новых записей, изменение и удаление существующих, а также просмотр информации о реализованной продукции. Модуль автоматически вычисляет размер индивидуальной скидки для каждого партнера на основе суммарного объема закупок, обеспечивает экспорт данных в текстовые форматы и содержит механизмы обработки исключительных ситуаций с выводом понятных пользователю сообщений.

Этапы создания программного продукта

Работа над приложением включала несколько последовательных этапов:

- Проектирование структуры данных – на данном этапе разрабатывалась логическая модель данных, приводимая к третьей нормальной форме. Определялся состав таблиц, устанавливались связи между ними, формировались первичные и внешние ключи. Графическое представление полученной структуры приведено на рисунке 2.
- Построение архитектуры приложения – инициализация проекта выполнялась на основе шаблона WPF Application. К основному проекту подключалась отдельная библиотека классов, в которую выносилась бизнес-логика.
- Тестирование разработанных компонентов – проводились модульные проверки отдельных частей системы и интеграционные испытания для подтверждения корректности взаимодействия всех модулей.
- Создание документации – исходный код снабжался пояснительными комментариями, формировалось описание функциональных

возможностей приложения и подготавливалось руководство для пользователя.

– Подготовка к распространению – создавались скрипты для развертывания базы данных и инструкции по установке приложения на целевых компьютерах.

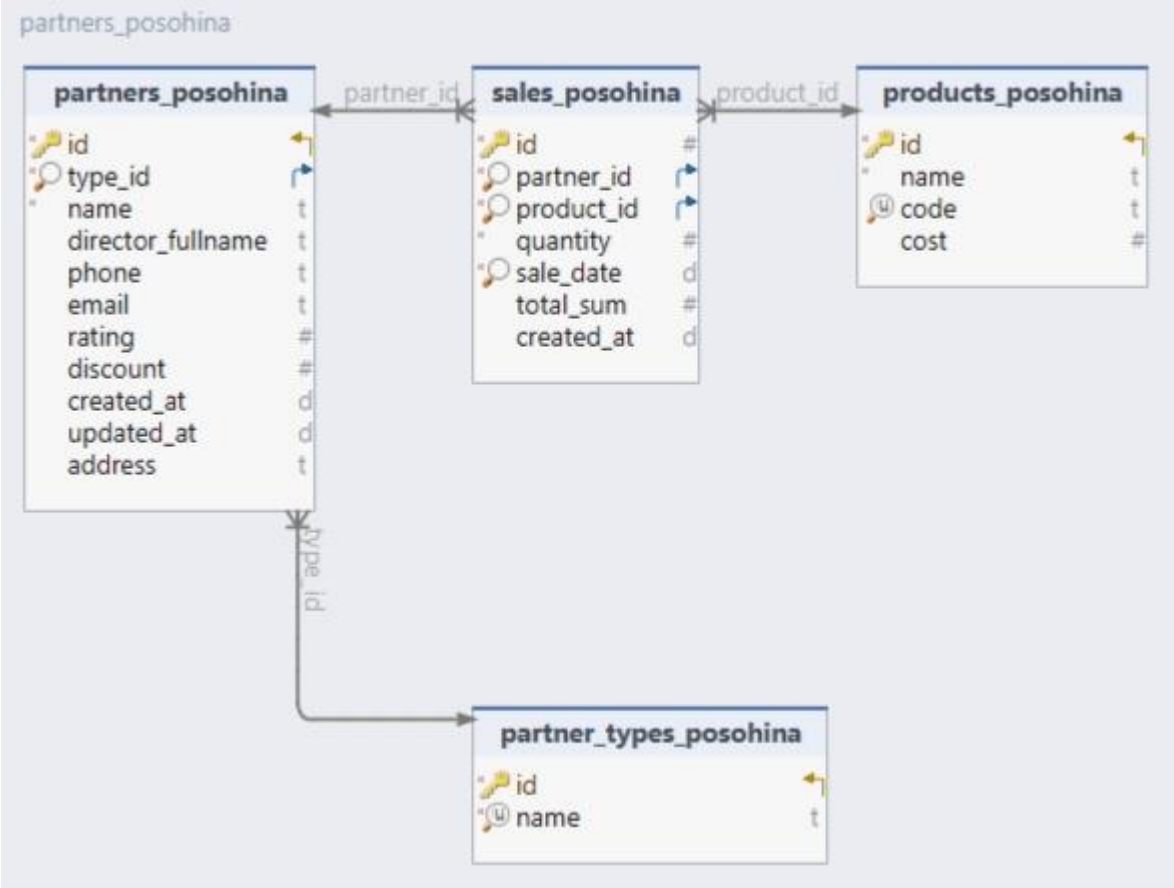

Рисунок 5 – Схема базы данных

Организация пользовательского интерфейса

Для обеспечения взаимодействия пользователя с разработанной системой создан комплект экранных форм, предназначенных для ввода, просмотра и корректировки информации:

– Главное окно приложения (изображено на рисунке 3) разделено на две рабочие зоны. Левая часть экрана отведена под список контрагентов, правая — под сведения о продажах, совершенных выбранным партнером. Управление приложением осуществляется через главное меню, а также через контекстное меню, вызываемое нажатием правой кнопки мыши на соответствующем элементе.

– Окно для работы с данными партнера (рисунок 4) используется как при добавлении нового контрагента, так и при изменении информации о существующем. В форме предусмотрены поля для ввода названия организации, выбора категории партнера из ниспадающего перечня, указания рейтинга (целое неотрицательное число), почтового адреса, фамилии, имени и отчества руководителя, контактного телефона и электронной почты.

– Окно регистрации продажи (рисунок 5) предназначено для фиксации факта реализации продукции. В данном диалоговом окне пользователь выбирает наименование товара из справочного списка, задает количество проданных единиц и указывает дату совершения операции. Итоговая стоимость продажи вычисляется автоматически на основе введенных параметров и отображается в предназначенном для этого поле.

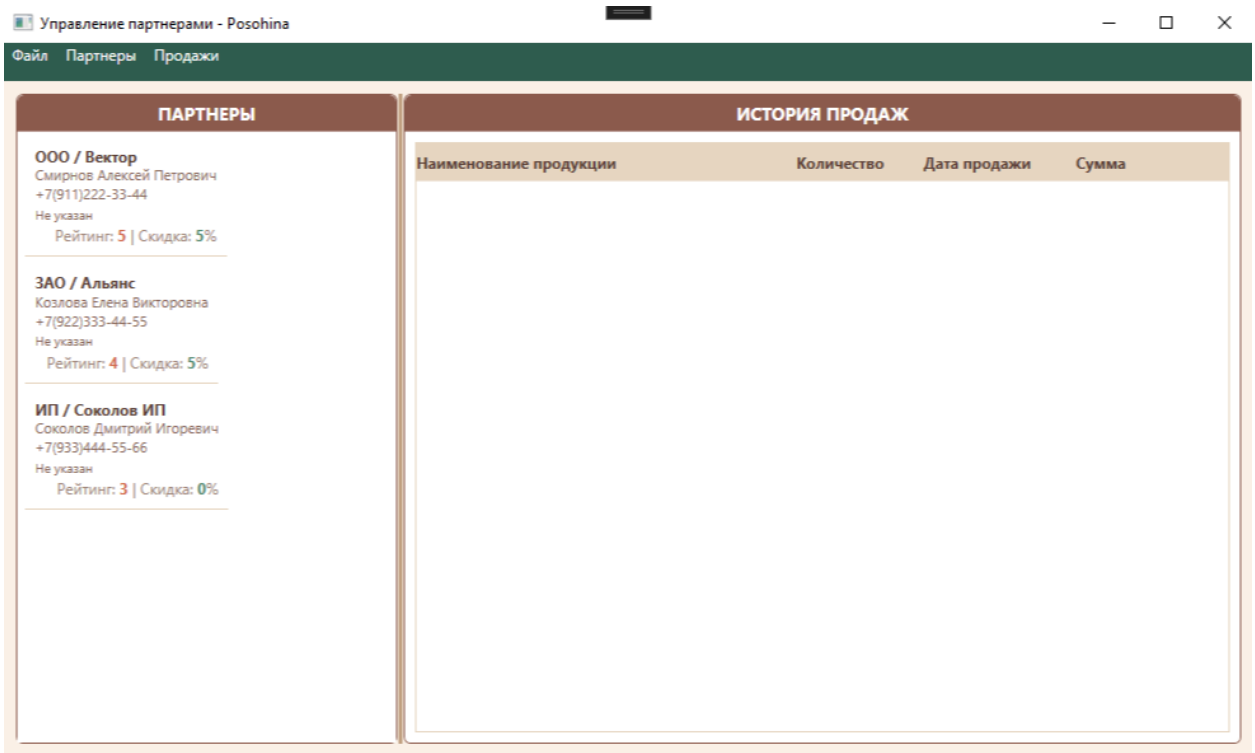

Рисунок 6 – Главное окно приложения

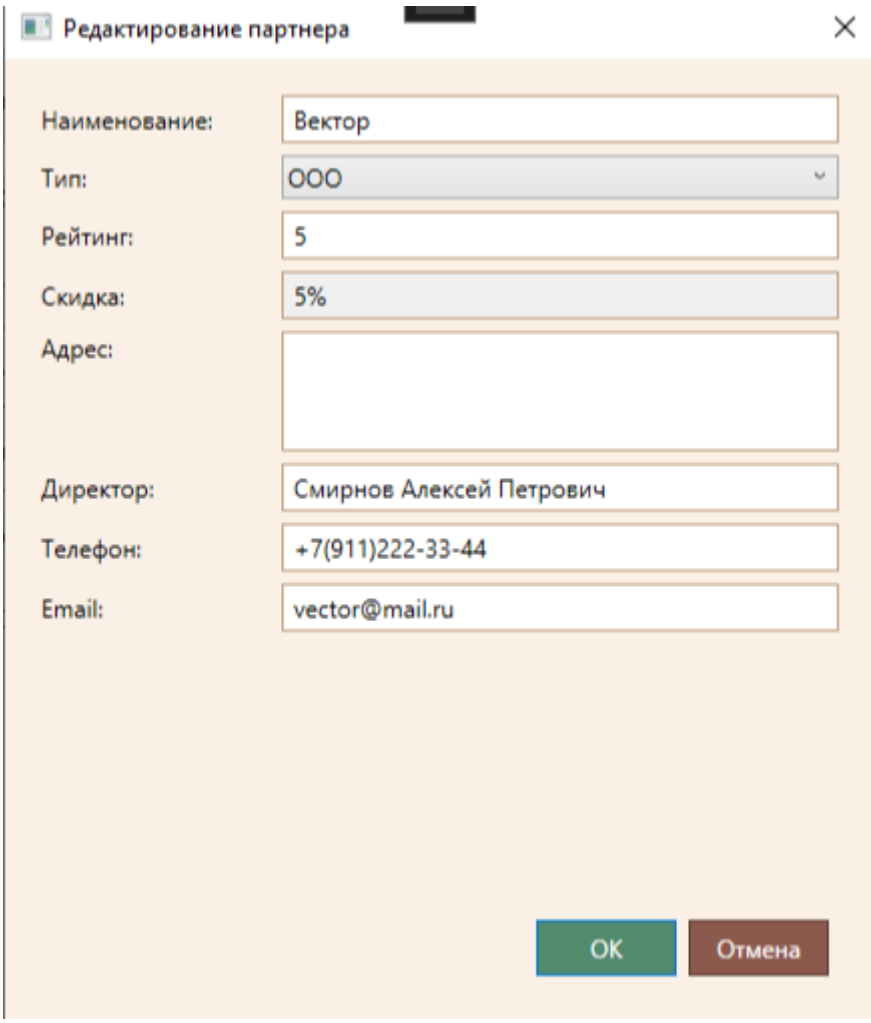

Рисунок 7 – Диалоговое окно добавления/редактирования партнера

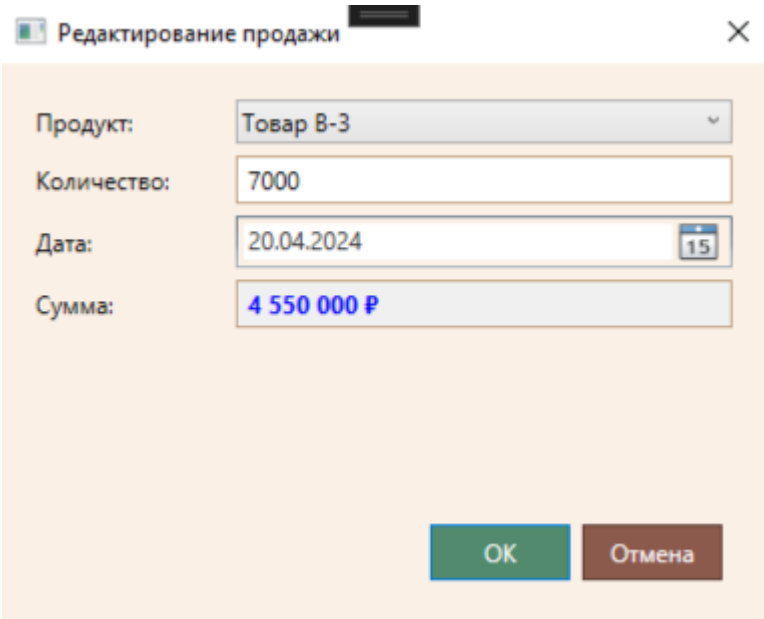

Рисунок 8 – Диалоговое окно добавления/редактирования продажи

Этапы разработки пользовательского интерфейса

Формирование графической оболочки приложения выполнялось в несколько последовательных этапов:

– Создание макетов – первоначально разрабатывались схематические изображения окон, пример которых представлен на рисунке 6. На этом этапе определялось расположение элементов управления, прорабатывалась структура главного окна с выделением двух информационных областей, а также проектировалась система навигации с использованием главного меню и контекстного меню.

– Оформление внешнего вида – на следующем этапе производилась настройка визуального представления элементов. Для интерфейса была выбрана строгая деловая цветовая схема с использованием темно-синих и серых тонов. Единство стиля достигалось путем определения общих ресурсов оформления, что обеспечивало идентичный внешний вид всех окон (конечный результат отображен на рисунке 3).

– Программная реализация навигации – заключительный этап включал разработку механизмов перехода между окнами и вызова контекстных меню для выполнения различных операций над выбранными записями.

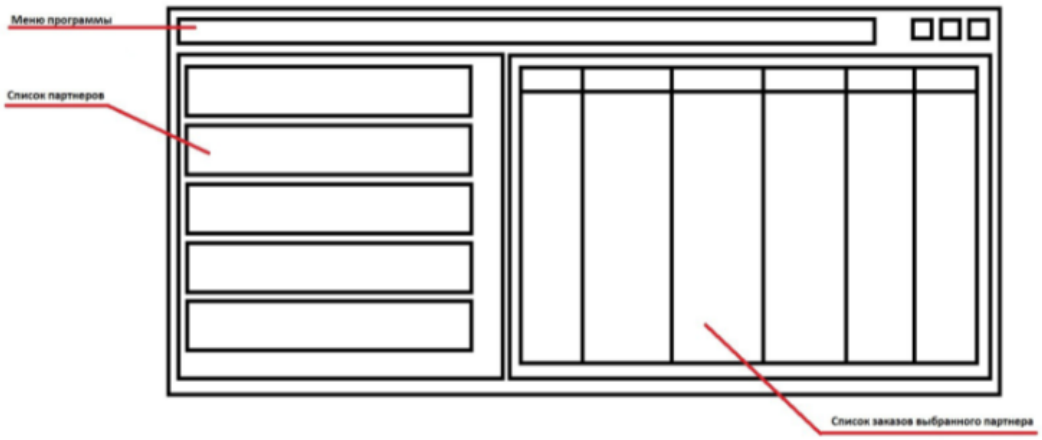

Рисунок 9 – Прототип интерфейса

3.1.3. Использование методов для получения кода с заданной функциональностью и степенью качества;

Меры по обеспечению качества и функциональности программного кода

В ходе создания программного продукта использовался ряд методов, направленных на достижение необходимого уровня функциональности и качества разрабатываемого решения:

- Применение архитектурного паттерна MVVM – использование модели Model-View-ViewModel позволило разделить логику представления и бизнес-логику (схема взаимодействия компонентов приведена на рисунке 7). Данный подход обеспечил удобство тестирования отдельных частей системы и возможность модификации пользовательского интерфейса без изменения внутренней логики приложения.
- Модульное построение кодовой базы – реализован принцип разделения ответственности: выделены самостоятельные компоненты для доступа к данным (библиотека классов), описания бизнес-сущностей (модели) и связующей логики (ViewModel). Такая организация кода обеспечила возможность повторного использования компонентов и упростила процесс тестирования.
- Соблюдение принципов SOLID – при проектировании иерархии классов учитывались требования единственной ответственности, расширяемости без изменения существующего кода и корректности наследования.
- Валидация пользовательского ввода – внедрена автоматическая проверка данных, вводимых пользователем, на уровне моделей и интерфейсных элементов, что позволило снизить количество ошибок при заполнении форм.
- Обработка исключительных ситуаций – для предотвращения аварийных остановок программы применялись конструкции перехвата исключений с последующим выводом информативных сообщений, помогающих пользователю понять причину возникшей проблемы.

Рисунок 10 – Схема паттерна MVVM

3.1.4. Инспектирование компонент программного обеспечения на предмет соответствия стандартам кодирования;

Оценка соответствия программного кода установленным требованиям

В процессе разработки выполнялся контроль качества исходного кода с применением различных методов проверки.

Используемые способы проверки:

- Экспертная оценка – проводился детальный разбор логической структуры компонентов для выявления скрытых дефектов и неэффективных решений.
- Автоматизированный анализ – применялись встроенные средства среды Visual Studio 2022, обеспечивающие постоянную проверку синтаксиса и обнаружение потенциальных ошибок непосредственно в процессе написания кода.
- Регламентированная проверка – контролировалось соблюдение принятых правил оформления и именования элементов программного кода.

Правила оформления, соблюдаемые при разработке:

- Для именования классов и открытых свойств использовался стиль PascalCase;
- Переменные и методы получали имена в стиле CamelCase;
- Наименования элементов выбирались таким образом, чтобы они однозначно отражали выполняемую функцию;
- В сложных для восприятия фрагментах код дополнялся пояснительными комментариями;

– Соблюдались единые правила форматирования, включая расстановку отступов и переносов строк.

Инструменты, задействованные для проверки:

– Встроенный анализатор управляемого кода, позволяющий обнаруживать скрытые проблемы и отклонения от рекомендуемых практик;

– Штатные средства Visual Studio 2022, обеспечивающие подсветку синтаксиса и оперативное выявление некорректных конструкций;

– Функции автоматического рефакторинга и форматирования, применявшиеся для приведения кода к единому стандарту.

3.1.5. Выполнение отладки программного модуля с использованием специализированных программных средств;

Процесс отладки программного модуля с использованием специализированных средств

Выявление и исправление ошибок в разработанном программном модуле осуществлялось с помощью отладочных инструментов, доступных в среде Visual Studio 2022. В ходе отладки выполнялись следующие действия:

– Мониторинг значений переменных – в окнах Autos и Locals контролировались текущие значения переменных, что позволяло оперативно обнаруживать некорректные или неожиданные изменения данных в процессе выполнения программы.

– Установка точек останова – в ключевых местах кода размещались маркеры останова, обеспечивающие приостановку выполнения программы для анализа состояния данных в конкретный момент времени.

– Диагностика исключений – при возникновении ошибок анализировалась цепочка вызовов (stack trace), что позволяло точно определить место и причину возникновения сбоя.

– Пошаговое выполнение кода – применялась трассировка с использованием клавиш F10 (пропуск вызываемых методов) и F11 (вход в вызываемые методы) для детального изучения последовательности выполняемых операций.

– Интерактивное тестирование – через окно Immediate выполнялись отдельные выражения и тестовые вызовы методов, что позволяло оперативно проверять предположения о работе фрагментов кода без необходимости повторного запуска всего приложения.

3.1.6. Осуществление разработки тестовых наборов и тестовых сценариев для программного обеспечения;

Создание тестовых наборов и сценариев для проверки программного продукта

Многоуровневая проверка работоспособности приложения

Тестирование разработанного приложения проводилось на различных уровнях:

– Проверка взаимодействия между модулями – оценивалась корректность взаимодействия различных компонентов приложения друг с другом, а также их совместная работа с базой данных.

– Автономное тестирование компонентов – выполнялась изолированная проверка отдельных модулей программы без их взаимодействия с остальными частями системы.

– Тестирование пользовательского интерфейса – проверялась правильность отображения элементов управления и адекватность реакции программы на действия пользователя.

Автоматизированное тестирование с использованием xUnit

Для автоматической проверки отдельных компонентов применялся фреймворк xUnit. Разработанные тесты охватывали следующие направления:

– проверка корректности расчета скидки для контрагента в зависимости от суммарного объема закупок;

– проверка правильности работы свойств в классах-моделях;

– проверка корректности отображения данных и обработки пустых значений в промежуточном слое ViewModel.

Результаты выполнения разработанных тестов представлены на рисунке 8. Среди проверяемых сценариев:

– отсутствие начисления скидки при нулевом объеме реализации;

– применение скидки в размере 5% при объеме продаж от 10 000 до 50 000 единиц;

– корректное преобразование наименования партнера для отображения в интерфейсе пользователя.

```
using Xunit;
using PosohinaLibrary.Models;

namespace PosohinaTests
{
    public class PartnerTests
    {
        [Fact]
        public void Partner_Discount_DefaultIsZero()
        {
            // Arrange
            var partner = new Partner();

            // Act
            var discount = partner.Discount;

            // Assert
            Assert.Equal(0, discount);
        }

        [Fact]
        public void Partner_CanSetDiscount()
        {
            // Arrange
            var partner = new Partner();

            // Act
            partner.Discount = 15;

            // Assert
            Assert.Equal(15, partner.Discount);
        }

        [Fact]
        public void Partner_Rating_CanBeNull()
        {
            // Arrange
            var partner = new Partner();

            // Act
            partner.Rating = null;

            // Assert
            Assert.Null(partner.Rating);
        }

        [Fact]
        public void Partner_Rating_CanBeSet()
        {
            // Arrange
            var partner = new Partner();

            // Act
            partner.Rating = 5;

            // Assert
            Assert.Equal(5, partner.Rating);
        }
    }
}
```

Рисунок 11 - Пример модульных тестов для работы с партнерами

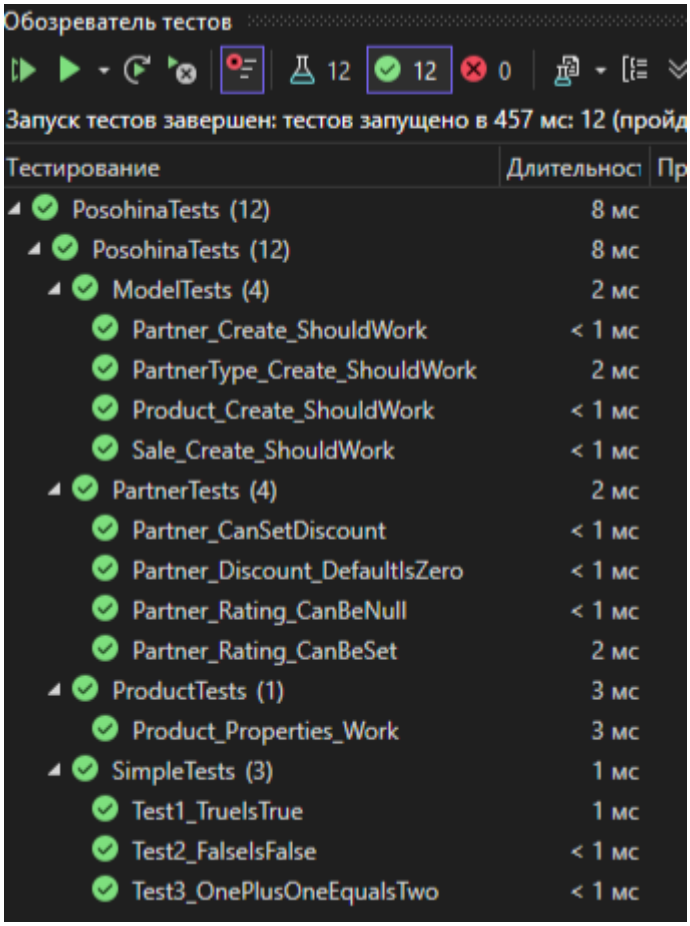

Рисунок 12 – Результаты выполнения unit-тестов

3.1.7. Интегрирование программных модулей;

Интеграция программных модулей в единое приложение

Процесс объединения разработанных компонентов в целостную систему выполнялся поэтапно. Первоначально были созданы два автономных компонента: модуль для работы с базой данных, реализованный с использованием Entity Framework Core, и модуль бизнес-логики, оформленный в виде отдельной библиотеки классов. На последующем этапе данные компоненты интегрировались с модулем представления, разработанным на базе WPF. Функцию связующего звена выполняли классы ViewModel, отвечающие за передачу данных между пользовательским интерфейсом и внутренней логикой приложения.

Организация доступа к данным реализована посредством внедрения экземпляра контекста AppDbContext в конструкторы соответствующих диалоговых окон, что обеспечило централизованное взаимодействие всех

частей приложения с базой данных. Для подтверждения корректности интеграции выполнялись сквозные пользовательские сценарии, включающие добавление нового контрагента, регистрацию продажи и последующее обновление информации в интерфейсе. Успешное выполнение данных сценариев подтвердило согласованность работы всех модулей системы.

3.1.8. Верификация и аттестация программного обеспечения;

Оценка соответствия программного продукта установленным требованиям

Процесс верификации в ходе разработки

На всех этапах создания системы осуществлялся контроль корректности разрабатываемых компонентов. Для проверки применялись следующие методы:

- Выполнение тестовых сценариев – проводилась проверка работоспособности программы с использованием заранее подготовленных тестов.
- Анализ программного кода – оценивалось соблюдение установленных правил оформления и структурирования исходного кода.
- Экспертиза архитектурных решений – анализировалась обоснованность выбранной архитектуры и логики функционирования отдельных компонентов системы.

Процесс аттестации готового продукта

После завершения всех работ проводилась итоговая проверка готовности приложения к эксплуатации. В ходе аттестации оценивались:

- корректность реакции программы на нештатные ситуации и ошибочные действия пользователя;
- стабильность функционирования при различных режимах работы;
- полнота реализации всех функций, предусмотренных техническим заданием;
- безошибочное выполнение всех пользовательских сценариев.

По результатам аттестации установлено, что разработанное приложение полностью соответствует предъявленным требованиям и готово к использованию в практической деятельности.

Заключение

В результате прохождения производственной практики с 09 по 17 марта 2026 г. были закреплены теоретические знания и расширены профессиональные умения.

По итогам практики получены следующие результаты:

1. Проанализирована проектная и техническая документация, на основе чего были формализованы требования к отдельным программным модулям, обеспечивающие их корректное взаимодействие в общей архитектуре ПО.
2. Осуществлено моделирование процесса разработки, что позволило оптимизировать этапы интеграции модулей и согласовать логику их работы.
3. Выполнена отладка программных модулей с использованием специализированных программных средств, что способствовало выявлению и устранению ошибок на ранних стадиях.
4. Разработаны тестовые наборы и тестовые сценарии для проверки функциональности и надежности программного обеспечения.

По окончании практики был получен практический опыт интеграции модулей в программное обеспечение; отладки программного модуля с использованием специализированных программных средств; разработки тестовых наборов и тестовых сценариев для программного обеспечения; инспектирования компонент программного обеспечения на предмет соответствия стандартам кодирования

Список используемых источников

1. ГОСТ 19.102-77. Единая система программной документации. Стадии разработки. – М.: Стандартинформ, 2010. – 4 с.

2. ГОСТ 34.601-90. Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания. – М.: Стандартинформ, 2009. – 8 с.

3. ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. – М.: Стандартинформ, 2011. – 104 с.

4. ГОСТ Р 51904-2002. Разработка программного обеспечения. Термины и определения. – М.: Госстандарт России, 2002. – 24 с.

5. Благодатских, В.А. Программная инженерия: учебник для вузов / В.А. Благодатских, В.А. Волнин, К.Ф. Поскакалов. – М.: Юрайт, 2022. – 415 с.

6. Терехов, А.А. Технология разработки программного обеспечения: учебное пособие / Под ред. А.А. Терехова. – М.: Форум, 2021. – 320 с.

7. Введение в Git [Электронный ресурс] / Git-scm.com. – Режим доступа: https://git-scm.com/book/ru/v2 (дата обращения: 13.03.2026).

8. Документация Microsoft Visual Studio 2022 [Электронный ресурс] / Microsoft Docs. – Режим доступа: https://docs.microsoft.com/ru-ru/visualstudio/ (дата обращения: 13.03.2026).

9. Руководство по C# и .NET [Электронный ресурс] / Metanit.com. – Режим

доступа: https://metanit.com/sharp/tutorial/ (дата обращения: 14.03.2026).

10. WPF. Руководство для начинающих [Электронный ресурс] / Metanit.com. – Режим доступа: https://metanit.com/sharp/wpf/ (дата обращения: 14.03.2026).

11. Документация Entity Framework Core [Электронный ресурс] / Microsoft Docs. – Режим доступа: https://docs.microsoft.com/ru-ru/ef/core/ (дата обращения: 14.03.2026).

12. PostgreSQL. Документация [Электронный ресурс] / Postgrespro.ru. – Режим доступа: https://postgrespro.ru/docs/postgresql/ (дата обращения: 14.03.2026).

13. Паттерн MVVM. Описание и примеры [Электронный ресурс] / Metanit.com. – Режим доступа: https://metanit.com/sharp/wpf/23.php (дата обращения: 15.03.2026).

14. Модульное тестирование с использованием xUnit [Электронный ресурс] / Microsoft Docs. – Режим доступа: https://docs.microsoft.com/ru-ru/dotnet/core/testing/unit-testing-with-dotnet-test (дата обращения: 15.03.2026).

15. Обработка исключений в C# [Электронный ресурс] / Metanit.com. – Режим доступа: https://metanit.com/sharp/tutorial/2.15.php (дата обращения: 15.03.2026).

16. Принципы SOLID в объектно-ориентированном программировании [Электронный ресурс] / Habr.com. – Режим

доступа: https://habr.com/ru/post/348286/ (дата обращения: 15.03.2026).

17. Троелсен, Э. Язык программирования C# 9.0 и платформа .NET 5 / Э. Троелсен, Д. Джепикс. – СПб.: Питер, 2022. – 848 с.

18. Рихтер, Д. CLR via C#. Программирование на платформе Microsoft .NET Framework / Д. Рихтер. – СПб.: Питер, 2020. – 896 с.

19. Методологии разработки программного обеспечения [Электронный ресурс] / Habr.com. – Режим доступа: https://habr.com/ru/post/495354/ (дата обращения: 12.03.2026).

Приложения

Приложение 1

Требования к программному обеспечению

Производственная компания «Мастер пол» выпускает напольные покрытия. Компания занимается производством и реализует свою продукцию через партнеров, которые доставляют продукцию компании до конечных потребителей.

Необходимо разработать подсистему для работы с партнерами компании, обеспечивающую следующий функционал:

– просмотр списка партнеров;
– добавление/редактирование данных о партнере;
– просмотр истории реализации продукции партнером.

При отображении истории реализации продукции должна выводиться следующая информация:

– наименование продукции;
– количество реализованной продукции;
– дата продажи.

При отображении и создании партнера должна использоваться следующая информация:

– наименование;
– тип партнера;
– рейтинг;
– адрес;
– ФИО директора;
– телефон;
– электронная почта.

Дополнительное требование: наличие возможности добавления/редактирования/удаления не только партнеров, но связанных с ними сущностей (тип партнера, продукция, история реализации продукции).

Исходя из предоставленных данных, для реализации системы необходимо в базе данных отобразить только четыре сущности:

- продукция;
- типы партнеров;
- партнеры;
- история реализации продукции партнерами.

Продукция представляет собой объект, у которого есть только артикул, который используется как уникальное значение для разграничения продукции между собой, и наименование. Так как в требованиях не указаны пункты о выводе другой информации, такой как стоимость продукции, изображение и прочее, то она не будет усмотрена в данной работе.

Тип партнера представляет собой объект, содержащий уникальное название и не более.

Партнер – комплексный объект, который кроме ранней представленной информации содержит и идентификатор своего типа.

История реализации продукции – объект, хранящий информацию о реализованной продукции, партнере, который реализовал продукцию, а также количество и дату продажи данной продукции.